

Lesson Plan : 22th May 2025

Students prepare for **Test 2 (Lab Test)**, covering both **Part A: Debugging** and **Part B: Programming**.

This lesson plan also includes sample exercises and a UML class diagram for a typical OOP-based problem using Association, Aggregation, and Composition.

Lesson Plan: Java Programming - Test 2 Preparation Class

Objective:

To prepare students for Test 2 (Lab Test) through targeted exercises and guidance covering Java fundamentals, ArrayLists, and Class Relationships (Association, Aggregation, Composition).

Test 2 Overview:

- **Mode:** Lab Test (Individual)
- **Parts:**
 - **Part A: Debugging**
 - Students will be given `.java` files with syntactic/semantic errors.
 - Goal: Fix the errors and get the program running correctly.
 - **Part B: Programming**
 - Students will be given a programming problem (with a UML class diagram).
 - Goal: Implement Java classes using the correct structure, relationships, and logic.
- **Focus Chapters:**

- **Chapter 2:** Classes and Objects (Constructors, Data Types, Methods)
 - **Chapter 5:** ArrayList
 - **Chapter 6:** Class Relationships (Association, Aggregation, Composition)
 - **Tools Provided:** `.java` skeleton files
 - **Submission Method:** Via Pen Drive (not through e-learning)
-

Exercise Samples

✓ **Part A: Debugging Exercise Sample**

Buggy File: `Patient.java`

Java

```
public class Patient {  
    private String name;  
    private int age  
  
    public void Patient(String name, double age){  
        name = name;  
        age = age;  
    }  
  
    public String displayDetails() {  
        System.out.println("Name: " + name)  
        System.out.println("Age: " + age);  
    }  
}
```

✓ **Student Task:** Identify and fix the bugs (e.g., missing semicolons, wrong variable assignment, etc.)

✓ Part B: Programming Exercise Sample

Problem Statement:

Implement a hospital management system using Association, **Aggregation** and **Composition** concepts. The system should include:

- **Hospital**
 - **Patient**
 - **Name** (firstName, lastName)
 - **Address** (street, city, postcode)
 - **Ward** (shared between Hospital and Patients)
-

```
+-----+
|  Hospital  |
+-----+
| - name: String      |
| - patients: ArrayList<Patient> |
+-----+
| +Hospital(name: String)      |
| +addPatient(p: Patient): void |
| +displayInfo(): void         |
+-----+
      1
      |
      | Association (uses)
      |
      *
      |
+-----+
| Patient |
+-----+
| - name: Name      | <-- Composition
| - address: Address | <-- Aggregation
+-----+
| +Patient(first, last, address) |
| +display(): void              |
+-----+
```

Name	Address
- firstName: String	- street: String
- lastName: String	- city: String
	- postcode: String
+getFullName(): String	+getFullAddress(): String

Unset

✅ **Student Task:** Implement the above classes in Java using appropriate relationships, constructors, and methods.

Key Reminders for Students:

- All files will be given in `.java` format.
- Focus on:
 - Proper use of constructors
 - Access modifiers
 - Relationship mapping (has-a)
 - Using `ArrayList` for multiple objects
- Submission **only via Pen Drive** at the end of the session.
- Ensure code compiles and runs before submitting.

